Server Internals

Kameswari Chebrolu

Department of CSE, IIT Bombay

https://www.codeitbro.in/backend-memes/

Background: Overall Outline

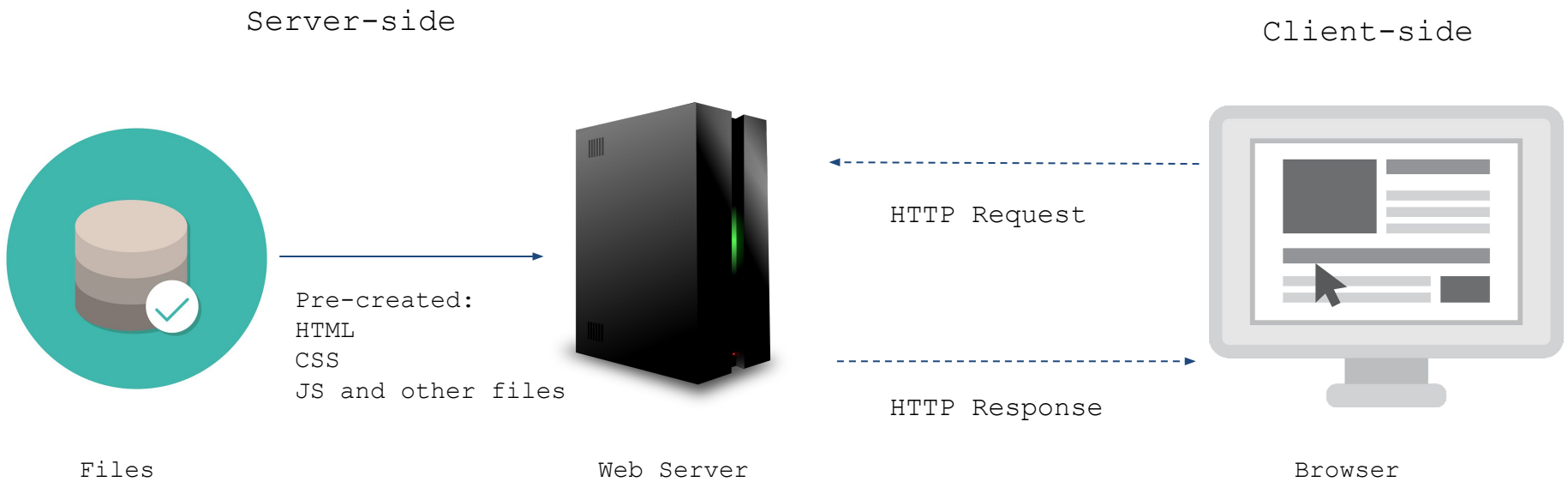
- ~~What constitutes a webpage?~~
- ~~What goes on inside a Browser?~~
  - ~~What standard security mechanisms implemented?~~
- ~~How do client and server communicate?~~
  - ~~HTTP/HTTPs protocol~~
  - ~~Session Management via cookies and tokens~~
- **How does a web server process requests and generate responses?**
  - **Static vs Dynamic content**

Outline

- Static Content
  - Web Server Software – Apache and Nginx
  - Web directory structure and permissions
- Dynamic Content
  - Sample Technology: PHP, Templates, Flask Framework, Web APIs, JSON data, Web Sockets
  - Web directory structure and permissions

Server Content

- Servers serve two types of content: Static and Dynamic
- Lets look at “Static” first
  - HTML files, image files, javascript files
  - Locate the requested file on file system and return unaltered in HTTP response


Server-side
Client-side
Pre-created:
HTML
CSS
JS and other files
HTTP Request
HTTP Response
Files
Web Server
Browser

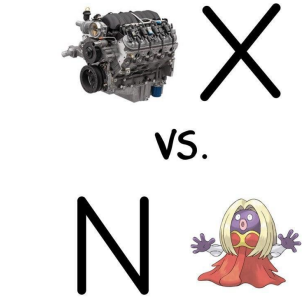
- Modern web servers may do some things extra!
  - Dynamically compress large resource files using gzip
    - Reduces bandwidth used in the response
  - Add caching headers in HTTP responses to instruct browser to cache
    - Decreases page load times in future if same file needed!

Web Server Software

- Plays a key role in managing connections and serving static content (dynamic as well)
- Apache HTTP Server (Apache): Oldest and most widely used open-source web server
  - Process driven architecture (not very memory efficient)
  - Configurability via files
    - main configuration file (apache2.conf)
    - .htaccess files in specific directories (override global settings)

- Rich ecosystem of modules that extend functionality
  - mod_ssl: Adds support for SSL/TLS encryption.
  - mod_rewrite: Enables powerful URL rewriting and redirection
    - http://example.com/product/123 rewritten to http://example.com/product.php?id=123
  - mod_proxy: Provides support for proxying requests to other servers
  - mod_auth: Handles authentication and access control

- Nginx: (pronounced "engine-x")
  - Configuration typically done via nginx.conf
  - Lightweight, high-performance web server
    - Event-driven architecture → handles concurrency very well
  - Also supports reverse proxying
  - Nginx often preferred for static content and as reverse proxy
  - Focuses on a small core feature set
    - Number of modules not as extensive as Apache's



https://preview.redd.it/nginx-how-do-you-pronounce-it-v0-c0t02u0j44b81.jpg

- Microsoft Internet Information Services (IIS): Supports various web technologies and integrates well with other Microsoft products

Web Directory Structure (Linux)

- Specifics vary based on distribution and server software
- Web Server Root: Base directory from which web server serves files (user requested)
  - HTML, CSS , Javascript files are placed here
  - For Apache, the default root directory is often /var/www/html
  - For Nginx, the default root directory is /usr/share/nginx/html
- Server Configuration Files: Configuration files for a web server are usually found in /etc
  - Apache configuration files might be in /etc/apache2
  - Nginx configuration files might be in /etc/nginx

- Logs: access logs and error logs, are typically found in /var/log
  - For Apache, logs might be in /var/log/apache2
  - For Nginx, logs might be in /var/log/nginx
- Virtual Hosts (if applicable): One web server can host multiple websites
  - Configurations for these virtual hosts may be stored in a subdirectory of the server's configuration directory
    - E.g., /etc/apache2/sites-available/ for Apache

Sample Apache Configuration

```
# Global configuration
ServerRoot "/etc/apache2"

# Listen on port 80
Listen 80

# Server-wide defaults
<Directory />
	Options FollowSymLinks
	AllowOverride None
	Require all denied
</Directory>

<Directory /var/www/>
	Options Indexes FollowSymLinks
	AllowOverride None
	Require all granted
</Directory>

# Logging
ErrorLog ${APACHE_LOG_DIR}/error.log
LogLevel warn
CustomLog ${APACHE_LOG_DIR}/access.log combined

# Include module configurations
IncludeOptional mods-enabled/*.load
IncludeOptional mods-enabled/*.conf

# Include additional directory configurations
IncludeOptional conf-enabled/*.conf

# Virtual hosts
IncludeOptional sites-enabled/*.conf
```

Explanation

- ServerRoot Directive: top-level directory where server's configuration files and assets are located
  - Set to "/etc/apache2"
- Listen Directive: port on which Apache listens for incoming connections
  - Set to default port 80

- <Directory> Directives: Define settings for specific directories
  - First <Directory> block sets options for the root directory ("/")
  - Second block for the default document root ("/var/www/")
  - Options: Specifies the default behavior for the specified directory
    - "FollowSymLinks" option allows symbolic links (symlinks) to be followed
    - Indexes" option allows server to generate and display directory listings for directories that don't have an index file (e.g. index.html)

- AllowOverride: Which directives can be overridden using .htaccess files
  - All: Allows all directives to be overridden by .htaccess files
  - None: Don’t allow directives to be overridden by .htaccess files
- Require: Access control rules
  - all: It refers to all users or clients, effectively meaning "everyone."
    - Require ip 192.168.1.0/24: Restricts access to only clients within the specified IP range
  - access is denied by default for / and explicitly allowed for document root

- Logging Section: Configures error and access logs
  - ${APACHE_LOG_DIR} is a variable often set to var/log/apache2
    - Defined in /etc/apache2/envvars
  - ErrorLog: Specifies the path to the error log file
  - LogLevel: Sets the verbosity level
  - CustomLog: Defines the format and location of the access log
    - “Combined" is one of a predefined log format

- Include Directives: Includes additional files
  - The .load files typically contain LoadModule directives, which load Apache modules.
  - The .conf files typically contain configuration settings
    - Related to the loaded modules (under mods-enabled)
    - Or other configurations (conf-enabled or sites-enabled for virtual hosting etc)

Permissions

- Installation of web servers like Apache typically require root permissions
  - installation involves placing files in system directories (e.g., /etc/, /usr/sbin/, etc.) and binding ports
- Configuration also requires root privileges
  - files are typically located in directories like /etc/apache2/, only writable by the root user

- Starting web server is normally done by a system service (e.g., systemd)
  - Needs root privileges
- Web server starts as root to bind to ports and read configuration files, load modules etc

- After initialization, if configured, servers can drop root privileges and runs as a non-privileged user (e.g. www-data)
  - Important for security!
  - Configuration: add below in config file (for Apache)

```
# User and Group directives
User www-data
Group www-data
```

- Main server process (still running as root) spawns child processes that handle incoming requests
- Child processes run under the www-data user
  - Ensures most of the server's operations (serving content, executing scripts, etc.) are done with limited privileges
  - Handles HTTP requests, serve content, execute CGI scripts, etc
  - Files and directories served by the web server are owned by www-data user
    - e.g., HTML, CSS, JavaScript, images in /var/www/html

Points to Note

- Suppose a user requests a file

  GET /users/index.html HTTP/1.1

  Host: www.example.com

  - File is at var/www/html/users/index.html
- Suppose a user requests a file

  GET ../../../etc/passswd HTTP/1.1

What happens?

- Apache configuration for <Directory /> will prevent this if set to “Require all denied”
- Note web-server can access this file if web-server runs as root (as opposed to www-data)
  - Attackers often try to trick web server to access such files on their behalf and serve files
  - All the more reason to drop privileges from root to www-data

Outline

- ~~Static Content~~
  - ~~Web Server Software – Apache and Nginx~~
  - ~~Web directory structure and permissions~~
- Dynamic Content
  - Sample Technology: PHP, Templates, Flask Framework, Web APIs, JSON data, Web Sockets
  - Web directory structure and permissions

Dynamic Content

- Add new item to inventory → painful to manually create a new product page
  - Repeat lot of code across each page
  - Any change to page structure → changes to many pages!
- Good to have an automated way to read from database and create product page!
- Other examples:
  - Display score/marks specific to logged in student
  - Process form submissions at server and provide feedback to user

- Dynamic Content: dynamically generated on the fly in response to user request
- Example:
  - Receive a HTTP GET Request for a product
  - Server determines the product ID
  - Fetches data from database
  - Constructs the HTML page for the response by inserting the data into a HTML template
    - Any changes need to be done in one place, in a single template, and not across many static pages!

Technology

- Server-Side Scripting Languages: Executed on server to create dynamic content
  - **PHP**, Python (Django), Ruby (Ruby on Rails), Node.js etc
- **Templates**: Help separate logic from presentation
  - Servers embed dynamic content within HTML templates
- Databases: Dynamic content generated via data stored and retrieved from databases
  - MySQL, PostgreSQL, MongoDB etc
  - Server-side scripts use queries (e.g. SQL) to interact with databases

- Web Frameworks: **Flask**, Django, Ruby on Rails, and Express.js provide structured web application development
  - Often based on Model-View-Controller (MVC)
  - Separate application into
    - models (data and business logic)
    - views (presentation)
    - controllers (handling user input and interaction)

- **Web APIs** (Application Programming Interfaces): Dynamic content can be sourced from external APIs
  - RESTful APIs are commonly used
- Real-Time Technologies:
  - **WebSockets**: provide a full-duplex communication channel between server and client
  - Server-Sent Events (SSE): a mechanism for sending updates from the server to client

Server-side Scripting: PHP

- PHP stands for Hypertext Preprocessor
- Embedded within HTML code itself
  - Allows developers to seamlessly mix server-side logic with HTML markup
  - Executed on the server, and the resulting HTML is sent to client

Workflow with PHP

1. **Client Request:** A user enters URL or clicks on a link
   - http://www.example.com/welcome.php?name=Chotu
2. **Web Server Handling:** Web server (e.g., Apache or Nginx) receives the request
3. Requested resource is a PHP file → web server hands over request to PHP interpreter

apache2.conf includes

LoadModule php_module modules/libphp.so

AddHandler application/x-httpd-php .php

LoadModule loads php module from given path and names it php_module

AddHandler associates given MIME type to files ending in .php

php_ module communicates with the PHP interpreter to execute the PHP script

Workflow with PHP

4. PHP Processing:

- PHP scripts often interact with databases (e.g., MySQL, PostgreSQL) to retrieve or update data.
- PHP generates dynamic content
  - Typically HTML, based on the logic in PHP script and any data retrieved from database

5. Response to Client:

- Generated content is sent back to server, which sends it as HTTP response to user's web browser

PHP interpreter processes the script, executes the PHP code and generates dynamic content (HTML) and sends it back to php_module

Example

```
<body>
    <h1>PHP Example</h1>

    <?php
    // Check if the 'name' parameter is present in the URL
    if (isset($_GET['name'])) {
    // Get the user's name from the 'name' parameter
    $userName = htmlspecialchars($_GET['name']);

    // Display a personalized greeting
    echo "<p>Welcome, {$userName}!</p>";
    } else {
    // Display a message if 'name' parameter is not provided
    echo "<p>Please provide your name in the URL.</p>";
    }
    ?>
</body>
```

http://www.example.com/welcome.php?name=Chotu

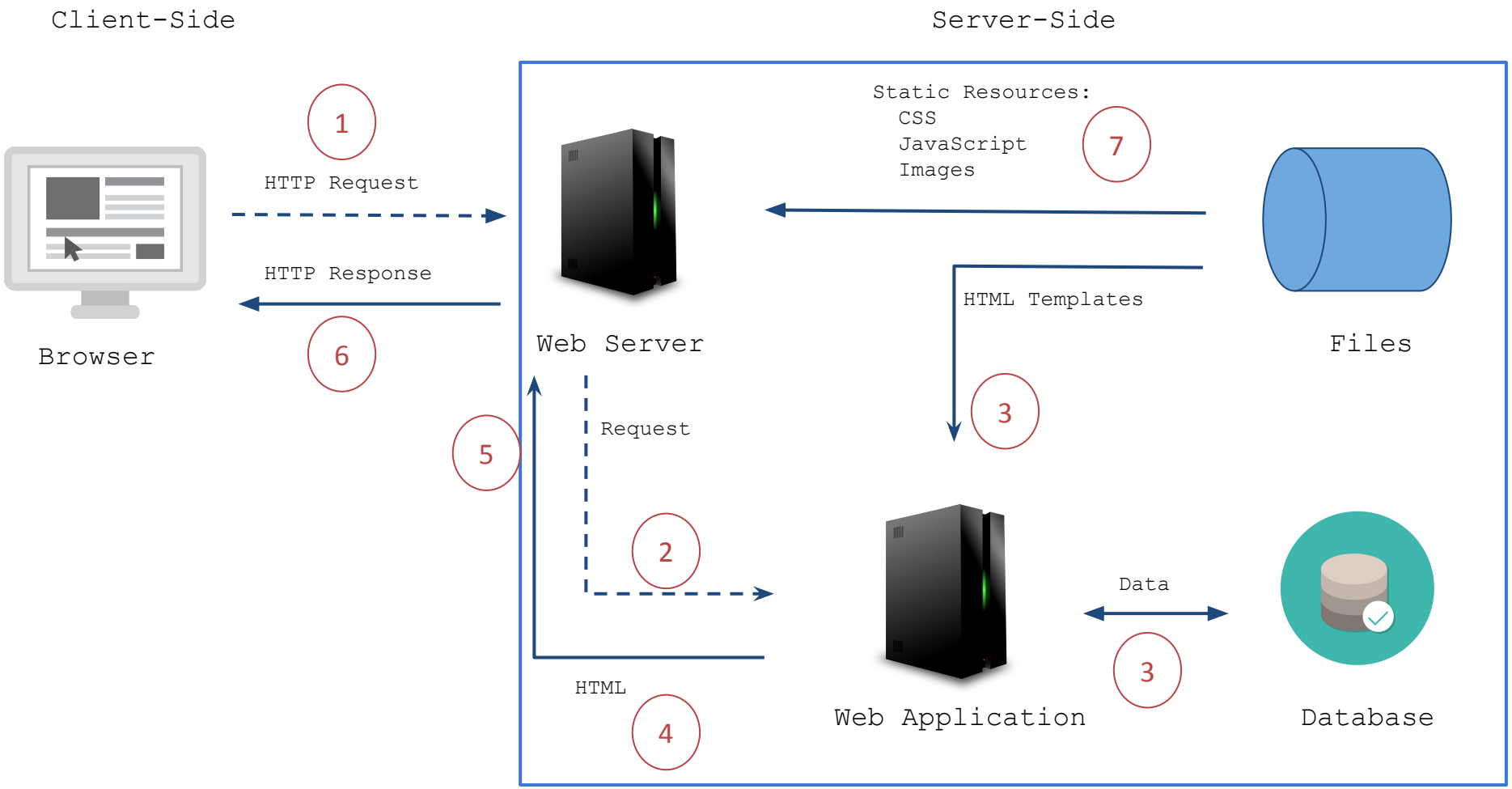
Question mark (?) indicates the beginning of a query string (set of key-value pairs)

- isset($_GET['name']): Checks if 'name' parameter is present in URL
- $_GET['name']: retrieve user's name
- htmlspecialchars() sanitizes the input
- Displays a personalized greeting using the user's name
- If 'name' parameter not provided, another message is shown

```
<body>
    <h1>PHP Example</h1>
    <p>Welcome, Chotu!</p>
</body>
```

Web Application Frameworks

- Software frameworks make it easier to write, maintain and scale web applications
  - Provide tons of tools and libraries that simplify common tasks
    - Routing URLs to appropriate handlers, template engines, interacting with databases, supporting sessions and user authorization, formatting output (e.g. HTML, JSON, XML), improving security etc
- Examples of popular web frameworks
  - Django (Python), Ruby on Rails (Ruby), Flask (Python), Spring Boot (Java)


Client-Side
Server-Side
Browser
1
HTTP Request
HTTP Response
6
Web Server
Static Resources:
CSS
JavaScript
Images
7
Files
HTML Templates
3
Request
5
2
HTML
4
Web Application
Data
3
Database

Workflow

- Web browser sends a HTTP GET request
  - E.g. https://www.example.com/product/123 or https://www.example.com/product?id=123
- Web Server detects request is "dynamic" and forwards it to Web Application for processing
  - How does it know where to forward?
    - Based on pattern matching rules defined in its configuration

Few Points to Note

- Apache is the web server
- Web application is say Flask
- Flask applications are typically run via Gunicorn. Why?
  - Flask provides a built-in development server (single-threaded) but not suitable for production environments
  - Gunicorn is a Python WSGI (Web Server Gateway Interface) HTTP server
    - Designed for production use, with more robust handling of concurrent requests and process management

```
<VirtualHost *:80>
    ServerName www.example.com

    # Proxy requests to Gunicorn backend
    ProxyPass / http://127.0.0.1:5000/
    ProxyPassReverse / http://127.0.0.1:5000/

    # Optional: Serve static files directly
    Alias /static/ /var/www/html/static/
    <Directory /var/www/html/static>
     Require all granted
    </Directory>


</VirtualHost>
```

Explanation:

- A client makes a request to http://www.example.com/
- Apache forwards the request to http://127.0.0.1:5000/ (Gunicorn server)
- Gunicorn passes it to an available worker process running the flask application flask (via IPC)

ProxyPassReverse:

- If Flask app responds with a redirect Location: http://127.0.0.1:5000/another-page
- With ProxyPassReverse / http://127.0.0.1:5000/, the redirect is rewritten by Apache to Location: http://www.example.com/another-page, making it accessible to the client

Alias

- A request to http://www.example.com/static/image.jpg will trigger the rule
- /var/www/html/static/ is the actual directory on the server's filesystem where the static files are stored

- Web Application identifies intent of URL
  - Gets required information from the database
  - Populates information into a **HTML template**
- Returns generated HTML to web browser (via the Web Server) along with status code
  - HTTP status code (200 for success; "404" to indicate product does not exist)

Example: Flask

```python
from flask import Flask, render_template, request

app = Flask(__name__)

# Route to the home page
@app.route('/')
def home():
    # Get the 'username' query parameter from the URL
    username = request.args.get('username', 'Guest')

    # Data to be passed to the template
    user = {'username': username}

    # Render the template 'index.html' with the provided data
    return render_template('index.html', user=user)

if __name__ == '__main__':
    app.run(debug=True)
```

- a route is defined for home page (`'/'`)
  - When a user accesses this page, `home` function is called
- request object from Flask is used to access the query parameters in URL
  - retrieves 'username' parameter, and if not present, defaults to 'Guest'
- A dictionary (`user`) is created
- `render_template` function is called to render 'index.html'
  - `user` data is passed on
- In the template (`index.html`), Jinja2 syntax (`{{ user.username }}`) is used to insert dynamic data

(Jinja2 is a template engine for Python programming language)

Template (index.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <title>Flask Template Example</title>
</head>
<body>
    <h1>Hello, {{ user.username }}!</h1>
    <p>This is a simple example of using templates with Flask.</p>
</body>
</html>
```

- Web Browser processes returned HTML
  - Can send separate requests for other CSS or JavaScript files
  - Web Server will return such static files directly

Web Directory Structure and Permissions

- Assume Flask Application with Gunicorn and Apache web server

```
my_project/
├── app.py          # Main Flask application file (entry point)
├── config.py       # Application configuration file (e.g., for DB, environment variables)
├── instance/       # Directory for sensitive configurations (optional, not tracked in version control)
├── static/               # Static files (CSS, JS, images)
├── templates/      # HTML templates for rendering views
├── wsgi.py         # WSGI entry point for Gunicorn to run the Flask app
├── logs/                 # Log files (optional, can be set in Gunicorn or Apache)
└── venv/                 # Python virtual environment (optional)
```

Folder of the Flask App

```
<VirtualHost *:80>
        ServerName www.example.com

        # Proxy requests to Gunicorn backend
        ProxyPass / http://127.0.0.1:5000/
        ProxyPassReverse / http://127.0.0.1:5000/

        # Serve static files from the project directory
        Alias /static /path/to/my_project/static

        <Directory /path/to/my_project/static>
        Require all granted
        </Directory>

        # Serve static files from /var/www/html
        Alias /images /var/www/html/images
        <Directory /var/www/html/images>
        Require all granted
        </Directory>


</VirtualHost>
```

- user account who deploys the application is deploy_user
- account that manages apache server is part of www-data group

my_project/static/	Directory	deploy_user	www-data	rwxr-x--- (750)
my_project/static/*	Files	deploy_user	www-data	rw-r----- (640)
my_project/logs/	Directory	deploy_user	www-data	rwxrwx--- (770)
my_project/logs/*	Files	deploy_user	www-data	rw-rw---- (660)

Outline

- ~~Static Content~~
  - ~~Web Server Software – Apache and Nginx~~
  - ~~Web directory structure and permissions~~
- Dynamic Content
  - Sample Technology: PHP, Templates, Flask Framework, **Web APIs, JSON data, Web Sockets**
  - Web directory structure and permissions

Web APIs

- API: Application Programming Interface
  - Set of rules and protocols that allows one application to interact with another
- Web API?
  - Enables communication and data exchange between web servers and also client-server
  - Typically use HTTP (Hypertext Transfer Protocol) as the communication protocol
  - Many web APIs follow REST principles

- Expose specific URLs (endpoints) to which clients send HTTP requests
  - Each endpoint represents a specific resource or functionality provided by the API
- Exchange data in standardized formats
  - JSON (JavaScript Object Notation) or XML (eXtensible Markup Language)

Example

Endpoint: http://api.example.com/users (This URL serves as the base for all user-related operations in the API)

```
GET /users HTTP/1.1
Host: api.example.com
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

[
{
"id": 1,
"username": "john_doe",
"email": "john.doe@example.com"
},
{
"id": 2,
"username": "jane_smith",
"email": "jane.smith@example.com"
}
]
```

```
GET /users/1 HTTP/1.1
Host: api.example.com
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
"id": 1,
"username": "john_doe",
"email": "john.doe@example.com"
}
```

```
POST /users HTTP/1.1
Host: api.example.com
Content-Type: application/json

{
  "username": "new_user",
  "email": "new.user@example.com"
}
```

Response

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "id": 3,
  "username": "new_user",
  "email": "new.user@example.com"
}
```

- For security,
  - Web APIs often require authentication
    - Clients must provide valid credentials (e.g., API keys, tokens) in the requests
  - Authorization also important
    - What actions or resources a particular user or application can access?
    - Ouath2.0 tokens and JWTs contain relevant info (user roles/permissions)

- Cross-Origin Resource Sharing (CORS) help web pages hosted on different domains to access web APIs
  - Relevant CORS headers specify which domains are allowed to make requests!
  - More in depth coverage later under attacks!

- Popular third-party web APIs: Twitter API, Google Maps API, GitHub API
  - Developers use these APIs to integrate third-party services into their applications

- As an aside: there are Browser APIs also
  - Extend functionality of the browser
    - E.g. Geolocation API returns coordinates of where browser is located
    - Web form API for input data validation
    - Web storage api for storing and retrieving data within browser
    - Web fetch api (saw as part of AJAX)

Data

- Exchange of data between a client and a server can occur in various formats
  - Already saw: html, css, js files; images in form of png, jpg; pdfs, docs etc
- APIs often involve transfer of structured data
  - JSON and XML are commonly used
  - Earlier in web's history, XML was in vogue, now JSON is more used!

https://preview.redd.it/vnzs43yknib61.jpg?auto=webp&s=b929e4e748d74606d901b50d8ac344346ad8a582

JavaScript Object Notation (JSON)

- A lightweight data interchange format
  - Easy for humans to read/write
  - Also for machines to parse and generate
- Data represented as key-value pairs
  - Supports data types such as strings, numbers, booleans, arrays, and objects
  - Data is enclosed in curly braces {}
  - Key-value pairs are separated by colons
  - Arrays are represented using square brackets []
  - Case-sensitive and whitespace-insensitive

```
{
  "name": "Ravi",
  "age": 22,
  "isStudent": true,
  "grades": [90, 85, 92],
  "address": {
      "city": "Mumbai",
      "zipcode": "400076"
  }
}
```

- MIME type: application/json
- Limitations:
  - no support for comments
  - no native support for binary data
  - Inability to represent circular references in objects
- More advanced features needed: use XML, Protocol Buffers, or GraphQL

Web Sockets

- WebSockets provide a full-duplex communication channel over a single, long-lived connection
  - Full-Duplex Communication: data can be sent in both directions simultaneously
    - Traditional HTTP follows a half-duplex model; client sends request, server responds!
  - Persistent Connection: connection remains open for as long as both client and server want

- WebSocket Protocol operates over a single, dedicated TCP connection.
  - Need to use ws:// and wss://
- Suitable for real-Time Applications
  - E.g. chat, online gaming, financial trading, live updates etc
- More details later under attacks!

https://httpcats.com/500.jpg

References

- https://developer.mozilla.org/en-US/docs/Learn/Server-side/First_steps/Introduction
- https://www.w3schools.com/php/
- https://www.geeksforgeeks.org/flask-tutorial/ and https://www.geeksforgeeks.org/getting-started-with-jinja-template/
- https://www.tutorialspoint.com/restful/index.htm (REST api)
- https://www.tutorialspoint.com/websockets/index.htm
- https://www.w3schools.com/js/js_json_intro.asp (json)